one asset, one lien.

A neutral first to file lien registry on Creditcoin. It witnesses pledges from lending protocols that never integrated with it, and refuses the second claim on the same collateral.

BUIDL CTC 2026 · RWA | singleton.unitynodes.com | 0xcD9017e3 · CC3 testnet, verified

A borrower pledges one deed twice, and nobody notices.

01

Harbor lends

1,000 against a tokenised deed. Non custodial, so the borrower keeps the token.

02

Meridian lends

750 against the same deed, an hour later. A different contract, no shared code or storage.

03

Both are live

Neither protocol can see the other's record. The collateral is now promised twice.

Custody prevents this mechanically. Everywhere custody is absent, it is live and unaddressed.

A contract cannot read a log. Not even its own.

`LOG0` through `LOG4` are write only. Receipts live in a trie execution never touches. `BLOCKHASH` reaches back 256 blocks.

So a neutral witness of somebody else's lending is either an off chain indexer, which is a trusted party and therefore not neutral, or a contract that consumes an inclusion proof of that log.

The second exists on Creditcoin and nowhere else.

BlockProver `0x0FD2` re-checks a merkle and continuity proof synchronously, inside the transaction that accepts the pledge.

This is a property of the machine, not a positioning claim.

Three steps, nobody trusted for any of them.

01

The protocol's own log

A lender emits its native event. NFTfi and Blur Blend are read on Ethereum mainnet, unmodified and unaware. Nothing is deployed there and nothing is asked of them.

02

BlockProver `0x0FD2`

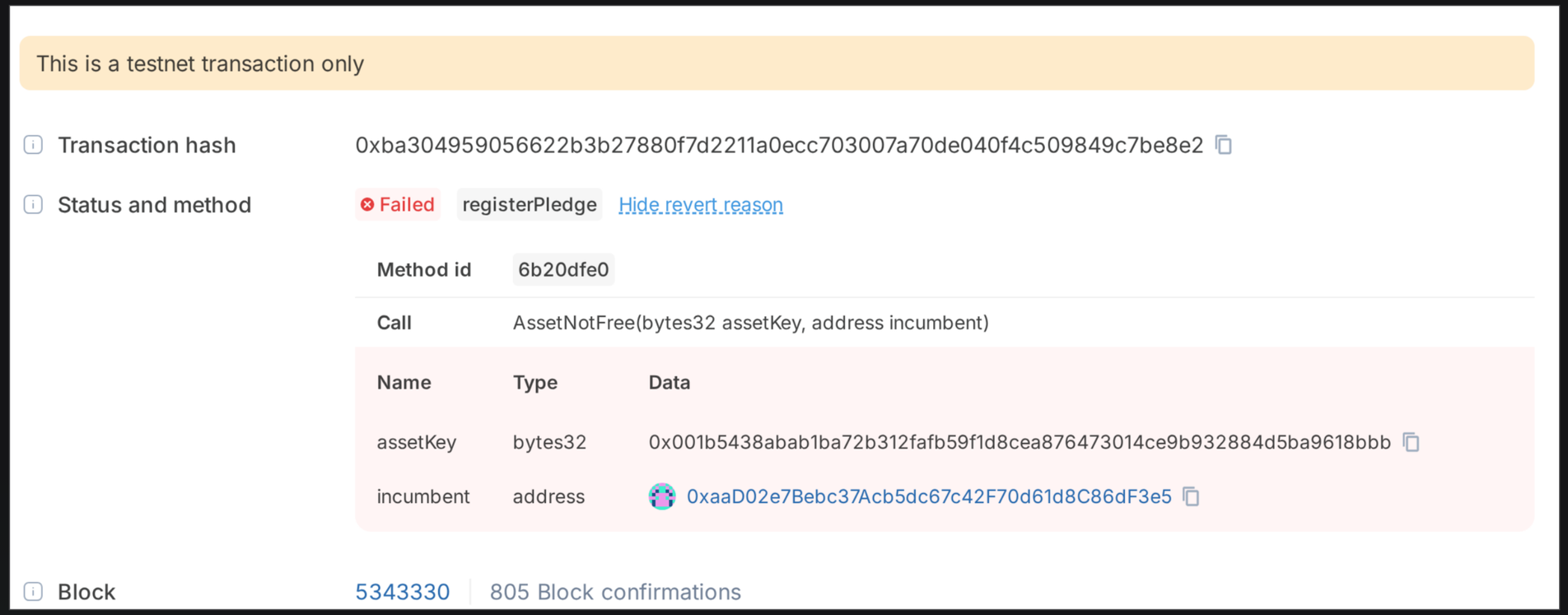
The precompile verifies the inclusion proof inside the accepting transaction, past a stated confirmation depth read from ChainInfo `0x0fD3`. The registry checks the source receipt status itself, because the precompile does not, and reads from AttestorStash `0x0FD4` how many bonded attestors that answer is worth.

03

The register

First to file is recorded and given a soulbound certificate. Anything second reverts. A nullifier per operation makes each proof spendable once.

The refusal is a failed transaction anybody can open.



Blockscout, CC3 testnet. The registry is verified there, so the custom error decodes into the asset key and the address of the lender who filed first.

Half the proofs read protocols that never heard of us.

NFTfi v3, loan 16928

Taken at mainnet block 25,506,517, repaid at 25,717,460. Real borrower, real NFT, 0.07 WETH.

Blur Blend, two liens

One closed by repayment, one seized after a failed auction. A lien ends in more than one way and an adapter that knew only repayment would leave seized liens on file forever.

Creditcoin attests Ethereum, so an existing loan can simply be read. No mainnet deployment, no funds there, no cooperation from either protocol.

And the part that does not flatter us.

Both of these escrow the collateral, which we checked on chain rather than assumed. So they prove the reader works and they cannot prove the fraud, because custody makes it impossible. The collision is on Sepolia, between two non-custodial lenders we wrote.

Depth, stated as calls rather than as adjectives.

In use

BlockProver `0x0FD2`,

both forms of verify

: one transaction for a single pledge, and an array against one shared continuity proof for a batch. ChainInfo `0x0fD3` for the attested tip and the chain key. AttestorStash `0x0FD4` for the bonded set behind that attestation. The deployed `EvmV1Decoder`.

Both

attested source chains: `get_supported_chains` reports two on CC3 testnet and both are read.

Measured, not estimated.

Four pledges filed one at a time cost 1,582,616 gas. The same four as one batch cost 989,237, which is 37.5 percent less. The batch constraint is in no document; we found it by breaking one.

And the explanation we first gave for that was wrong.

On Creditcoin a contract's code size is charged on every call: the same trivial call costs 22,318 gas against an account with no code and 189,996 against a 12.8 KB registry, about 13 gas per byte across three deployments. So 566,034 of the saving is entering the contract three fewer times, and only 27,345 is the shared continuity proof. We found this chasing 27,650 gas that appeared on an operation we had not touched.

Refused, with reasons

SubstrateTransfer moves CTC and a register holds none. The signature verifiers want Substrate keys and the evidence here is an inclusion proof. Staking and the legacy loan pallets are not reachable from Solidity at all. Writability is not shipped.

Refused after measuring

`is_height_attested` works, and on both live chains it is exactly the comparison the finality guard already makes: true up to the attested tip, false one block past it, no gaps below, genesis zero. Spending a call to be told what `height ⩽ tip` says is not depth. The full list with reasons is in `docs/SURFACES.md`.

Every cross chain record inherits a quorum. Nobody writes it down.

The set is not a constant, and we checked rather than assumed

The precompile answers at historical heights, so this is the readable history and not an argument for it. Sepolia went 0 to 1 to 6 to 7 between May and July. Ethereum went 0 to 1 to 3 to 4, and was on the supported chain list for a month before a single attestor was bonded for it.

A record made while seven stood behind it and a record made while two did are stored identically by every bridge, oracle and message layer shipped so far. After the fact there is no way to ask how much was standing.

So the quorum is part of the record

The count, the bond and the attested tip are read inside the transaction that accepts a proof, stored with the lien and with every refusal, and emitted in a log that outlives the record when the lien is released and the record is deleted.

And it is enforced.

Below a stated floor the registry stops filing anything for that chain. Entry only, never exit, so no attestor rotation can strand an asset already on file. We raised the floor above the live set on purpose and the chain refused a real proof: `QuorumTooThin(1, 7, 8)`, transaction `0x9ecf965f`, still there.

It points at us too.

Ethereum carries four attestors against our floor of three. Two deregistrations halt the mainnet half of this demo, because the guard refuses below the floor and not at it. We did not lower the floor to two to make that go away, because three is the smallest set where no single attestor is a majority.

Two independent reviews in one day. Both found real bugs.

01

Receipt poisoning

A borrower could attach a decoy log carrying the registry's own event signature and make their genuine pledge permanently unregisterable.

02

The same mistake, one door along

The fix scoped log counting to the chosen emitter, but the emitter was still inferred from the receipt, which the borrower orders. One log from any other allowlisted protocol suppressed the pledge again.

03

What changed

The relayer now names the emitter and the log index. The registry searches for nothing. A batch pledge became supported rather than refused, and mainnet gas fell from 716k to 634k.

Also corrected: a caveat claiming an administrator cannot fabricate, which was false, and the vacuous test that backed it. 98 tests, every attack kept as a regression.

Said before anybody has to ask.

A positive record

An asset the register calls free is one nobody registered here, not one nobody pledged. Attestcoin proves a transaction happened, never that one did not.

The adapter is trusted

The proof decides whether a log exists; the adapter decides what it means. Bounded interpretation of a proven fact, in under a hundred and thirty lines of pure code each, not unbounded trust in existence.

The key is free here only

`keccak(chainKey, token, tokenId)` names an on chain asset with no agreement between protocols. Off chain invoices are the larger market and the harder key.

And the one we would rather say than have found. For unique collateral the non-custodial market is empty: the one protocol that shipped in-wallet encumbrance financed a single loan, and no lender uses any lockable token standard. We think that is a consequence rather than an objection.

Lenders take possession because there is no register to check

, so possession is the fallback a missing register forces, which makes this the piece that has to exist first. That is an argument and not a proof, which is why it sits on this slide.

Running, and checkable without asking us for anything.

16

inclusion proofs on CC3 testnet, every hash published

2 / 2

attested source chains read, Sepolia and Ethereum mainnet

98

tests, including both reviews kept as regressions

0

wallets, backends and indexers between the page and the chain

`singleton.unitynodes.com` `/demo, 1:53, voiced and captioned` `github.com/UnityNodes/singleton`

`0xcD9017e3C541cAF973987E23e02694111C25032C`